

JSP

Java Server Pages



Del Cliente al Servidor

Hasta ahora trabajamos del lado del cliente:

- ▶ **HTML** → estructura
- ▶ **CSS** → presentación
- ▶ **JavaScript** → interacción

Todo esto ocurre en el navegador.

Pero una aplicación real necesita más. Hay tareas que requieren otro entorno:

- ▶ guardar datos
- ▶ autenticar usuarios
- ▶ validar información sensible
- ▶ aplicar reglas de negocio

Para eso existe
el **servidor**.

¿Por qué Java?

Java es un lenguaje ampliamente utilizado en entornos empresariales para construir aplicaciones del lado del servidor.

Su adopción histórica se relaciona con varias características:

- ▶ orientación a objetos
- ▶ tipado fuerte
- ▶ portabilidad mediante JVM
- ▶ herramientas maduras
- ▶ buen soporte para concurrencia

Esto lo convirtió en una opción frecuente para sistemas robustos y de larga vida útil.



¿Qué es Jakarta EE?

Jakarta EE (anteriormente Java EE) es un **conjunto de especificaciones y tecnologías orientadas al desarrollo de aplicaciones empresariales** en Java.

No es un lenguaje nuevo.

Java sigue siendo el lenguaje.

Jakarta EE es la plataforma que organiza herramientas para construir aplicaciones robustas del lado del servidor.

Incluye tecnologías para resolver problemas frecuentes como aplicaciones Web, servicios REST y SOAP, persistencia de datos, seguridad, transacciones, etc.



Ciclo básico de una solicitud

Cuando un usuario interactúa con una aplicación Web suele ocurrir lo siguiente:

- ▶ el navegador envía una solicitud (request)
- ▶ el servidor la recibe
- ▶ procesa datos o reglas
- ▶ genera una respuesta (response)
- ▶ el navegador interpreta el resultado.

Este intercambio puede repetirse miles de veces por minuto en sistemas reales.

La Web funciona mediante comunicación entre componentes separados.

¿Qué es JSP?

JSP (Java Server Pages) es una tecnología que **genera páginas dinámicas desde el servidor utilizando HTML combinado con componentes Java.**

Su objetivo fue simplificar la construcción de interfaces Web dinámicas.

En lugar de enviar una página estática, el servidor puede construir contenido según:

- ▶ usuario autenticado
- ▶ datos consultados
- ▶ fecha actual
- ▶ resultados de negocio

El navegador recibe HTML final, no código Java.

Idea importante sobre JSP

Una página JSP:

- ▶ se escribe como archivo .jsp
- ▶ contiene HTML y elementos dinámicos
- ▶ el servidor la procesa antes de enviarla

El navegador no ejecuta JSP.

El navegador recibe HTML final.



¿Qué ocurre al pedir una página JSP?

- ▶ El cliente solicita la página.
- ▶ El servidor recibe el pedido.
- ▶ La página JSP se transforma en un servlet.
- ▶ Se ejecuta código Java.
- ▶ Se genera HTML.
- ▶ Se responde al navegador.

¿Qué es un Servlet?

Un Servlet es **una clase Java preparada para responder solicitudes HTTP**. Puede encargarse de:

- ▶ recibir parámetros
- ▶ validar datos
- ▶ consultar información
- ▶ decidir navegación
- ▶ construir respuestas

Normalmente se implementan métodos como:

- ▶ doGet()
- ▶ doPost()

El servlet representa lógica del lado del servidor.

Contenedor de Servlets

Los servlets no funcionan solos.

Necesitan un **entorno de ejecución** llamado **contenedor**, por ejemplo:

- ▶ Apache Tomcat
- ▶ Jetty
- ▶ GlassFish

Este componente administra:

- ▶ carga de clases
- ▶ ciclo de vida
- ▶ routing de requests
- ▶ sesiones
- ▶ seguridad básica

Características de las páginas JSP

- ▶ **Separación entre presentación y lógica:** Permiten combinar HTML con procesamiento dinámico de forma más organizada.
- ▶ **Etiquetas reutilizables (Tag Libraries):** Incorporan etiquetas que encapsulan lógica sin escribir Java directamente en la vista.
- ▶ **Uso de plantillas y fragmentos comunes:** Facilitan reutilizar encabezados, menús, pies de página y estructuras repetidas.
- ▶ **Trabajo conjunto con Servlets:** Habitualmente JSP se ocupa de la interfaz, mientras los servlets manejan la lógica de negocio.

Directivas JSP

Las directivas son elementos que **proporcionan instrucciones globales al contenedor de servlets** sobre cómo debe manejar ciertos aspectos de las páginas JSP.

Es decir, que son **esenciales para el control y la administración de cómo las páginas JSP se comportan y se integran** con otros componentes de la aplicación. Permiten a los desarrolladores manejar configuraciones complejas y dependencias de manera organizada y reutilizable, facilitando la gestión del código y la mantenibilidad de la aplicación.

Afectan la fase de traducción de la página JSP a un servlet Java. Existen principalmente tres tipos de directivas en JSP: **page**, **include**, y **taglib**.

Directivas JSP: Page

La directiva **page** **define atributos específicos de la página JSP**, como el tipo de contenido, el juego de caracteres, los scripts de importación y otras configuraciones de la página. Entre sus atributos comunes encontramos:

- ▶ **language**: Especifica el lenguaje de scripting utilizado en la página, generalmente es Java.
- ▶ **contentType**: Define el tipo MIME del contenido que la página generará, por ejemplo, text/html; charset=UTF-8.
- ▶ **import**: Permite importar paquetes o clases Java que se utilizarán en la página.
- ▶ **session**: Indica si la página utiliza o no una sesión HTTP (true o false).

Directivas JSP: Page (Cont.)

Siguiendo con los atributos comunes:

- ▶ **isELIgnored**: Especifica si las expresiones de “Lenguaje de Expresión” (EL) se deben ignorar o no.
- ▶ **errorPage**: Define una URL a una página que manejará cualquier excepción lanzada y no capturada en la página actual.
- ▶ **isErrorPage**: Indica si la página actual se utiliza como una página de error.

```
<%@ page contentType="text/html; charset=UTF-8"  
language="java" import="java.util.*, java.text.*"  
session="false" %>
```

Directivas JSP: Include

La directiva **include** **incluye estáticamente el contenido de otros archivos** en la página JSP durante la traducción del JSP a servlet. Esto es útil para reutilizar componentes como encabezados, pies de página, o bloques de código comunes en múltiples páginas.

```
<%@ include file="header.jsp" %>  
... contenido de la página ...  
<%@ include file="footer.jsp" %>
```

Directivas JSP: Taglib

La directiva **taglib** **declara una biblioteca de etiquetas personalizadas que se utilizará en la página.**

Proporciona una forma de incluir funcionalidades personalizadas sin utilizar Java directamente en la página JSP. Entre sus atributos comunes encontramos:

- ▶ **uri:** La URI asociada a la biblioteca de etiquetas, que identifica la biblioteca.
- ▶ **prefix:** El prefijo que se utiliza para identificar las etiquetas de la biblioteca en la página.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core"
prefix="c" %>
```

```
<c:out value="¡Hola Mundo!! :)" />
```

Scriptlets en JSP

Los scriptlets son **bloques de código Java que se insertan dentro de una página JSP**.

Aunque los scriptlets ofrecen una forma flexible de integrar la lógica de programación con contenido HTML, **su uso es generalmente desaconsejado** en favor de prácticas más modernas que promueven una mejor separación entre la lógica de presentación y la lógica de negocio.

Un scriptlet se escribe dentro de un archivo JSP usando las delimitaciones `<% y %>`. Todo el código que se coloca dentro de estas etiquetas se ejecuta cuando la página JSP se procesa en el servidor.

Scriptlets en JSP (Cont.)

```
<%
```

```
    String name = "Visitante";
```

```
    if (request.getParameter("name") != null) {
```

```
        name = request.getParameter("name");
```

```
    }
```

```
%>
```

```
<html>
```

```
<head><title>Ejemplo de Scriptlet</title></head>
```

```
<body>
```

```
    <h1>Hola, <%= name %></h1>
```

```
</body>
```

```
</html>
```

Objetos implícitos en JSP

Los objetos implícitos en JSP **son instancias de varias clases importantes que el contenedor de servlets automáticamente pone a disposición** de los desarrolladores en cualquier página JSP. Estos objetos cubren una amplia gama de funcionalidades, incluyendo la gestión de solicitudes, respuestas, sesiones, y contexto de la aplicación. A continuación, nombraremos algunos de los objetos implícitos más utilizados en JSP:

- ▶ **request:** Es una instancia de `HttpServletRequest` y proporciona información sobre la solicitud del cliente, incluyendo parámetros, encabezados y cookies.
- ▶ **response:** Es una instancia de `HttpServletResponse` y se utiliza para controlar todo lo relacionado con enviar una respuesta al cliente, como establecer encabezados o el código de estado.

Objetos implícitos en JSP (Cont.)

Objetos implícitos (cont.):

- ▶ **session:** Representa la sesión HTTP y permite a los desarrolladores almacenar objetos que persisten a través de múltiples solicitudes entre el mismo cliente y el servidor.
- ▶ **application:** Es una instancia de ServletContext y se utiliza para compartir datos entre todas las páginas, solicitudes y usuarios.
- ▶ **out:** Es una instancia de JspWriter utilizada para enviar contenido en la respuesta. Es un buffer que ayuda a enviar la salida de texto a los clientes de forma eficiente.
- ▶ **config:** Proporciona información de configuración asociada con el servlet que corresponde a esta JSP.

Objetos implícitos en JSP (Cont.)

Objetos implícitos (cont.):

- ▶ **pageContext:** Proporciona acceso a todas las características del contexto de esta página y también a los diversos ámbitos.
- ▶ **page:** Este es el servlet subyacente en el que se ejecuta la JSP, equivalente a `this` en Java.
- ▶ **exception:** Solo está disponible en las páginas que han sido designadas como páginas de error mediante la directiva *isErrorPage*. Contiene la excepción lanzada en la página JSP.

Qué debería quedar en claro

- ▶ Java suele utilizarse en backend
- ▶ JSP genera contenido dinámico
- ▶ Servlet procesa solicitudes
- ▶ El contenedor administra ejecución
- ▶ Cliente y servidor colaboran



